

画像を入力とするモデルの基礎

画像を扱うモデル



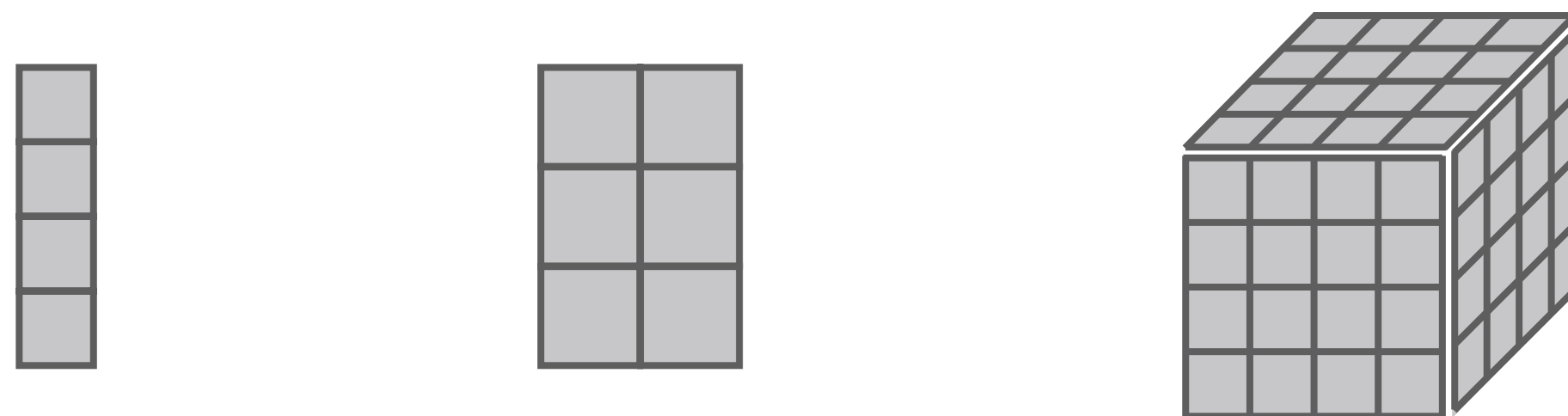
画像データに特化したモデルにはどんなものがあるか

- 非構造化データなので、元の数値の並びから高次の情報を抽出する必要がある
- 画像データを扱う上で、いくつかの重要な事前知識がある
 - 数値が1次元的に並んでおらず、長方形や直方体などの形に並んでいる
 - 写っている物体の位置が多少ズレても意味が変わらないなどの不変性をもつ

講義の前提知識＞ノーテーションの注意

画像などを扱うときは，テンソルという用語がよく出てくる

- 多次元的に数値を並べたものを**テンソル (tensor)** という

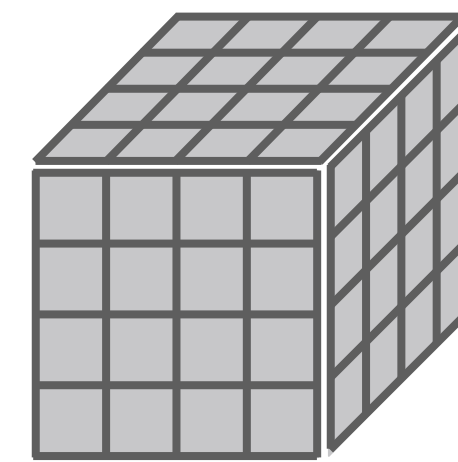


- テンソルの各軸のことを**モード (mode)** という
 - モードの個数は**次数 (order)** という
 - 1次のテンソル＝縦ベクトルや横ベクトル，2次のテンソル＝行列
- 各モードの要素数を**次元 (dimension)** という

講義の前提知識 > ノーテーションの注意

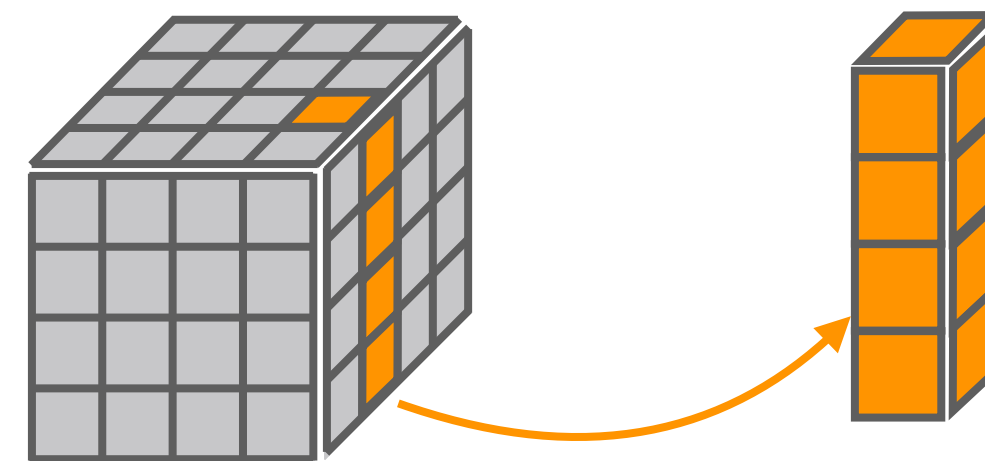
テンソルに関する用語も導入しておく

- テンソル a の各モードの次元がそれぞれ $d_1, d_2, \dots, d_r$ であるとき, a は形状 (shape) が $(d_1, d_2, \dots, d_r)$ であるという



形状 (4,4,4) のテンソル

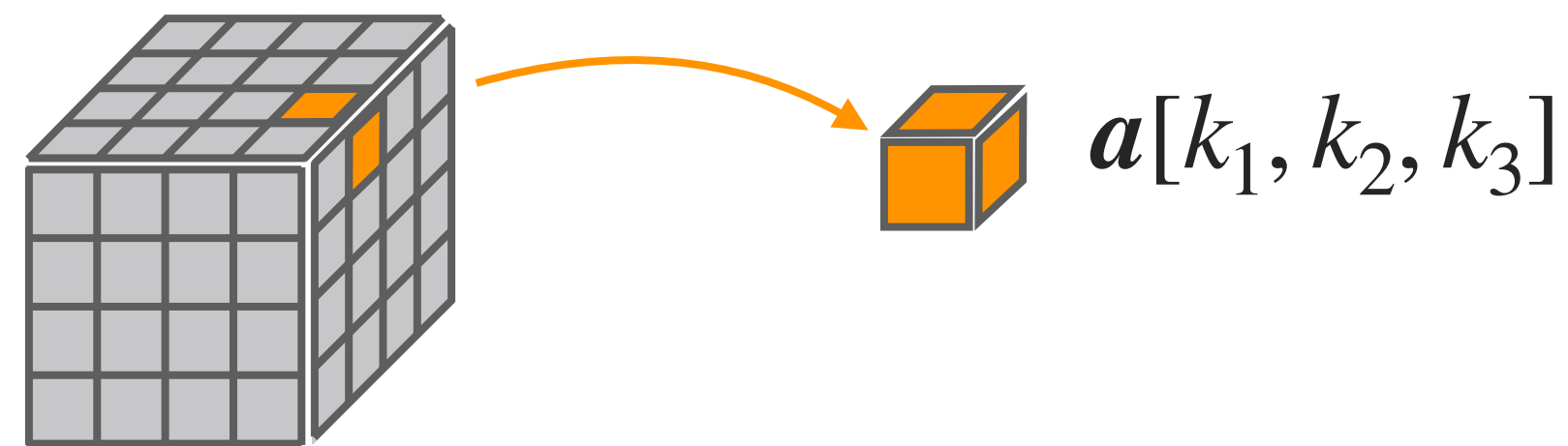
- 部分テンソル：テンソルの一部を切り出したテンソルのこと



講義の前提知識 > ノーテーションの注意

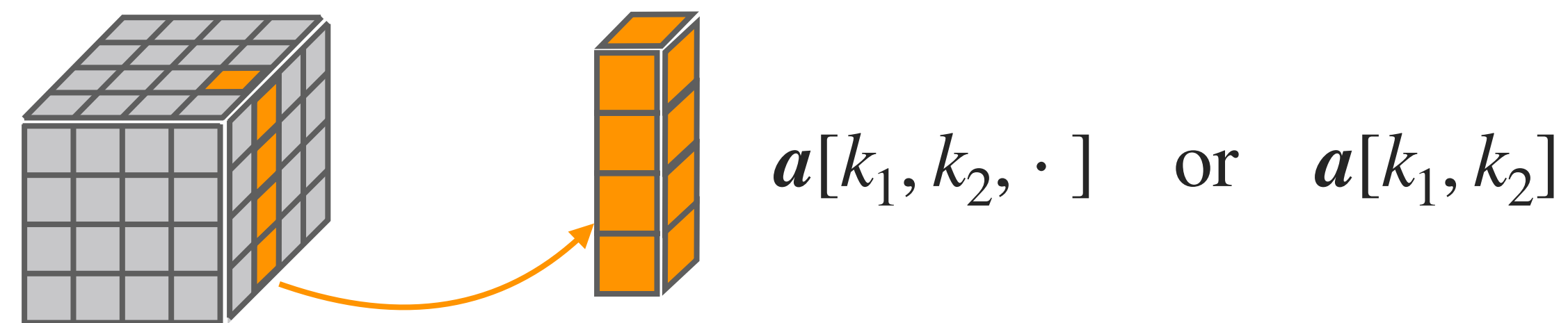
この講義では、テンソルの成分を表すために次の記号を使う

- 次数 r のテンソル a に対して、その第 $(k_1, k_2, \dots, k_r)$ 成分を $a[k_1, k_2, \dots, k_r]$ と表す.



- 一部のモードの添字 (index) だけを指定した場合、部分テンソルを表す

- 指定しない添字は「 $\cdot$ 」で表す



- 最初の数個のモードの添字のみを指定する場合、後ろ側の「 $\cdot$ 」は省略する

画像を扱うモデル＞位置関係の事前知識

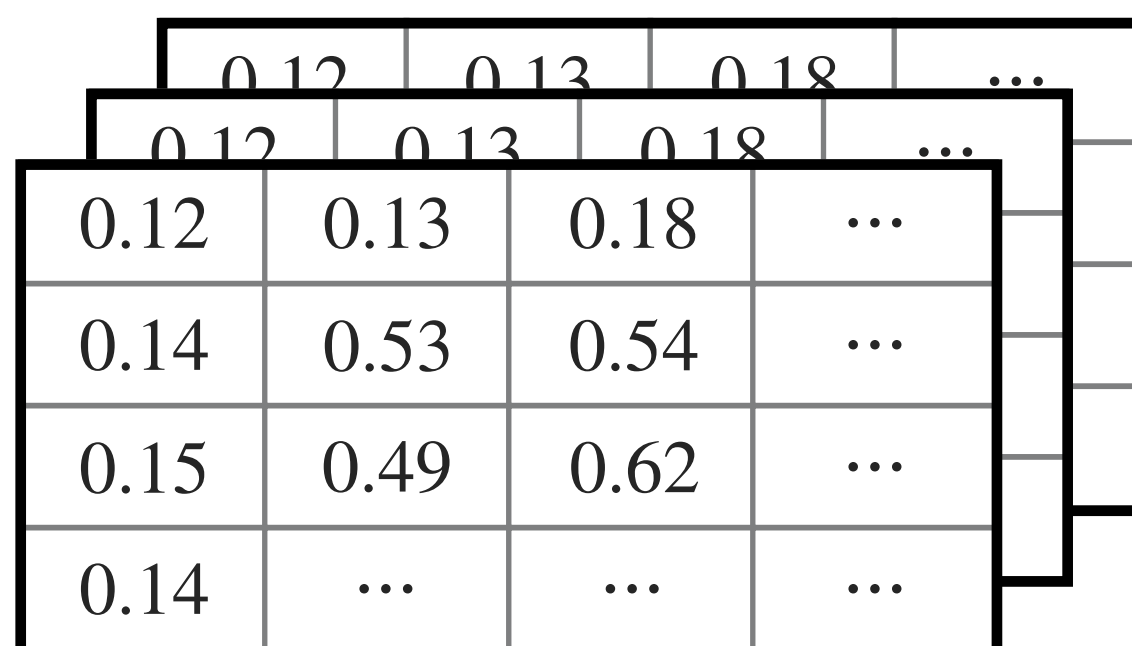
画像データに共通する事前知識がある

- 2次元・3次元的な数値の配置が意味を持つ
 - 縦横のつながりに意味があるので、値の並び順を変えると情報が失われる

	0.12	0.13	0.18	...
	0.12	0.13	0.18	...
0.12	0.13	0.18	...	
0.14	0.53	0.54	...	
0.15	0.49	0.62	...	
0.14	...	...	...	

画像を扱うモデル＞位置関係の事前知識＞考慮しない場合

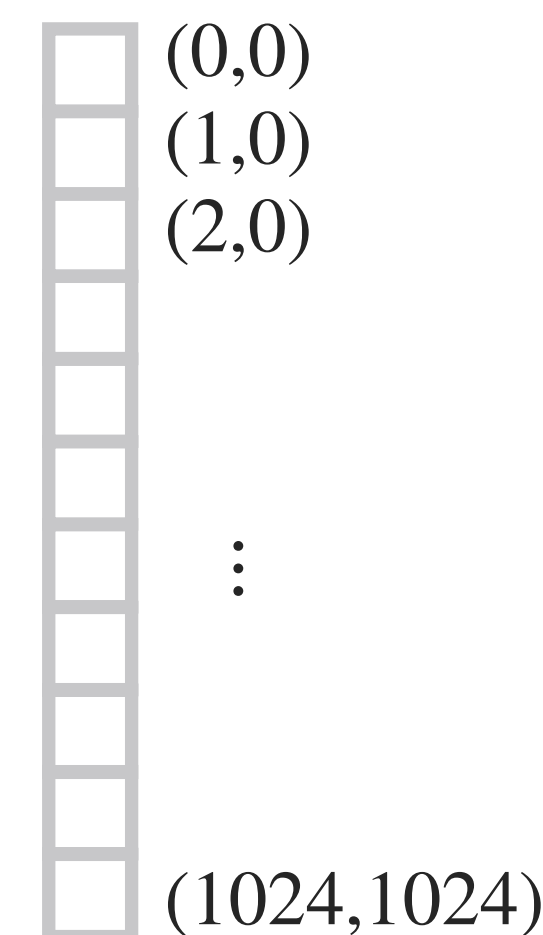
多次元に並んだ値を，1次元ベクトルに直して扱うことはできそうにも思える



0.12	0.13	0.18	...
0.14	0.53	0.54	...
0.15	0.49	0.62	...
0.14	...	...	...

画像データは多次元
(縦x横xRGBなど)

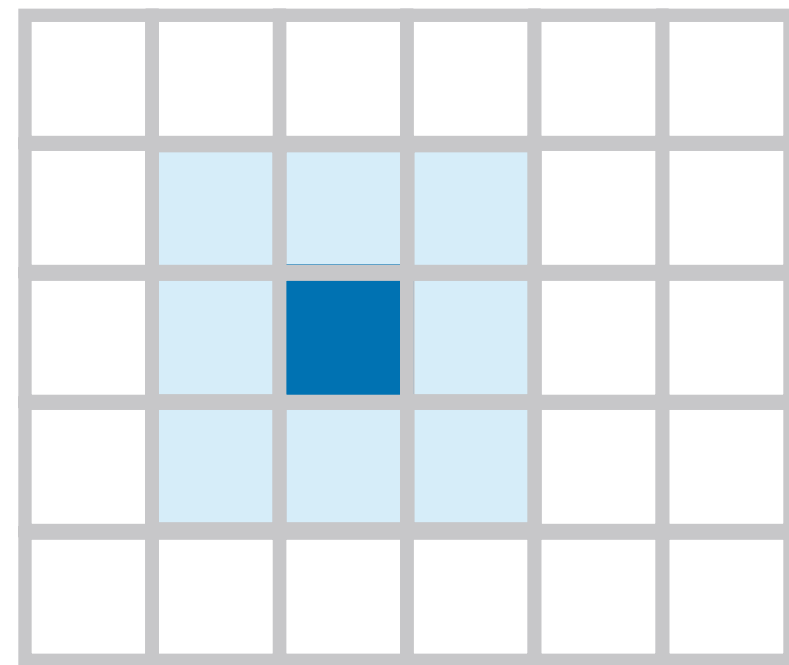
形状だけ1次元に変更



1次元化
(フラット化, flattening, と呼ぶ)

画像を扱うモデル＞位置関係の事前知識

ところが、この方法では、元の隣接関係が失われてしまう



本来はピクセルの隣接関係に
重要な意味がある



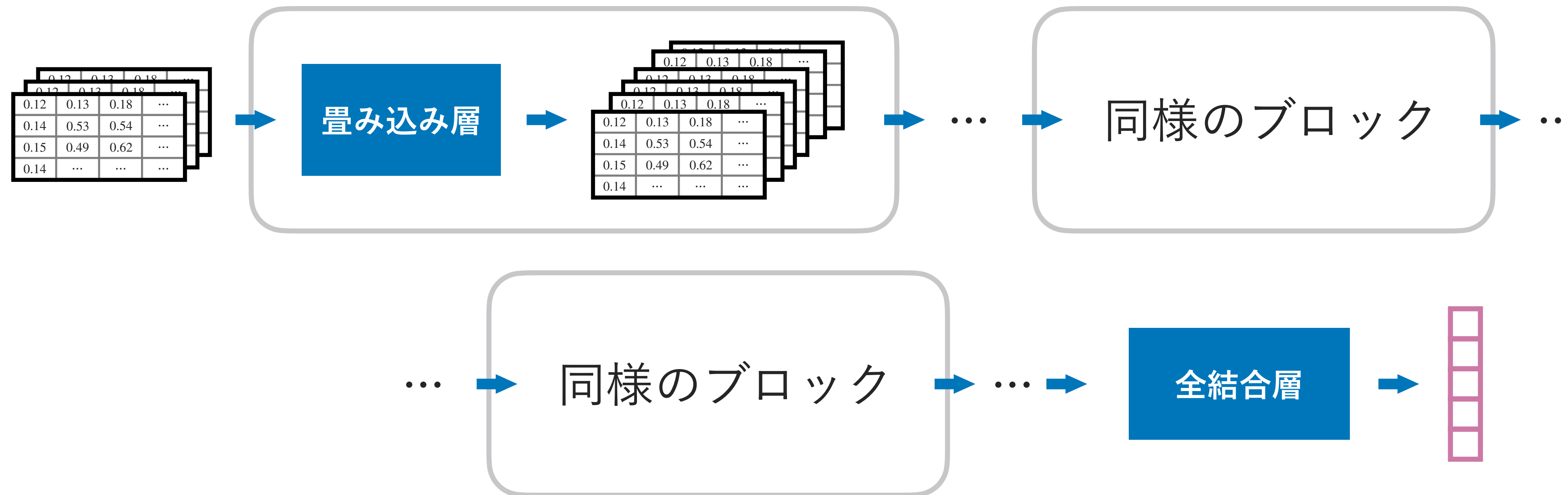
一次元化で構造が失われる

- できれば、ピクセルの位置関係を活かせるモデルを用いたい（事前知識）

画像を扱うモデル＞畳み込み層

画像に関する事前知識を活かしたモデル

- 畳み込みニューラルネットワーク (convolutional neural network, CNN)



画像を扱うモデル＞畳み込み層

画像の処理に適したモデルの構成要素

- 畳み込み（convolution）という演算を使う
 - まずカーネルという行列を用意（例：3x3カーネル， $a, \dots, i$ はパラメーター）

a	b	c
d	e	f
g	h	i

- 画像の各座標で，カーネルと同じ形状の範囲に対して，内積を計算する

2	3	3			
4	8	0			
0	0	5			

a	b	c
d	e	f
g	h	i



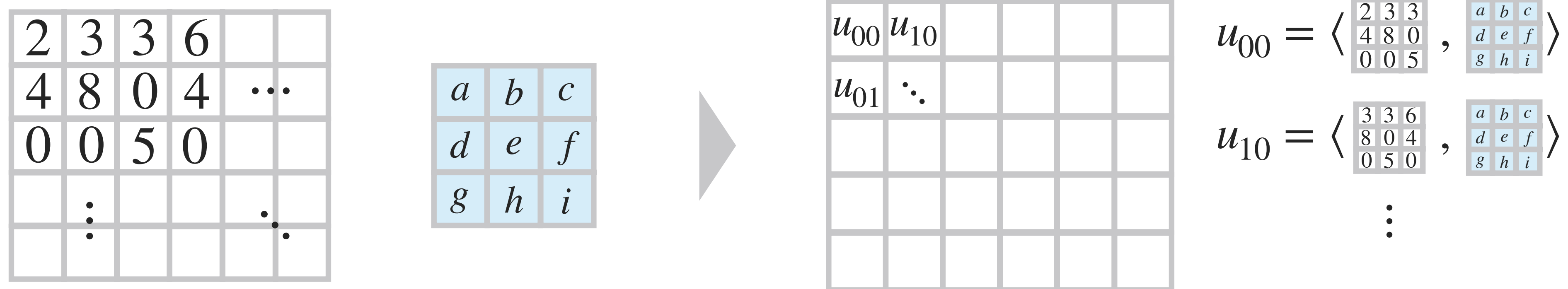
座標(0,0)に対しては，

$$u_{00} = 2a + 3b + 3c + 4d + 8e + 5i$$

画像を扱うモデル＞畳み込み層

この計算を，座標を変えながら行ってゆき，全ての座標について行う

- 座標ごとに，この内積の計算を行う



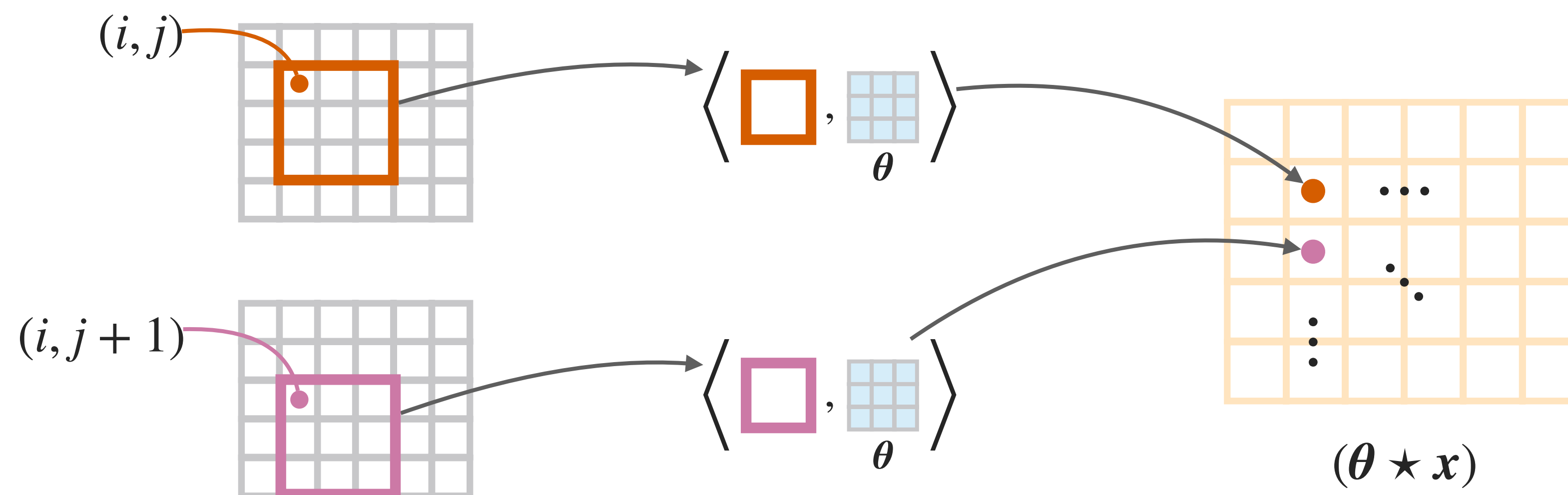
- この処理の結果は，画像に似た，多次元的に数値が並んだものになる

【補足】 実際には，すべての座標で計算するとは限らず，この内積計算を行う対象の座標は，数ピクセルずつ間隔を空けることもある。これは，解像度の高い画像では，全ての座標で律儀に計算をしても，隣り合ったピクセル同士ではどのみち値が似通っていて，実質的に無駄な計算を沢山することになってしまうため。

画像を扱うモデル＞畳み込み層

画像の上をカーネルでスキャンしていくような処理になっている

$$(\theta \star x)[i, j] := \sum_{q=0}^Q \sum_{s=0}^S \theta[q, s] \cdot x[i + q, j + s]$$



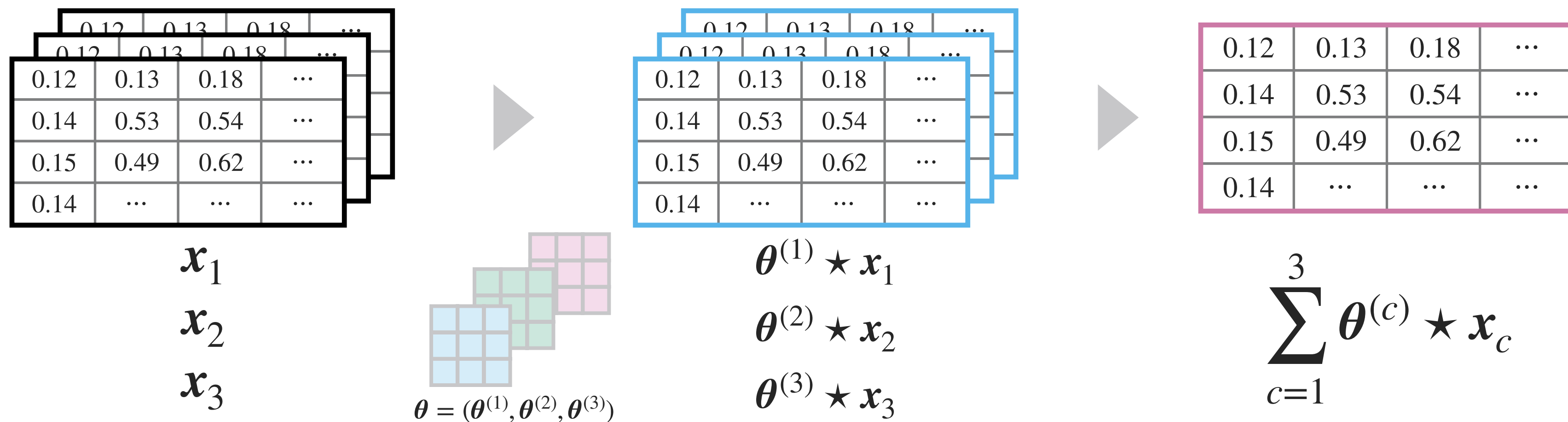
- この処理は、畳み込み（convolution）と呼ばれる（数学では相互相関という）

【用語】 数学の通常用語では、この $x \star \theta$ は「 x と θ の畳み込み」ではなく、相互相関（cross-correlation）と呼ばれる。畳み込みと呼ばれる演算を用いて表すこともできるため、機械学習の文脈では畳み込みと呼ばれることがある。

画像を扱うモデル＞畳み込み層

入力が複数チャネルある場合は、カーネルもそれに合わせて多次元化

- 入力が複数チャネルある場合、それぞれのカーネルを用意（例：RGB画像）



- この場合まで考慮して、次のように定義しておく（ C は入力のチャネル数）

$$\text{Conv}(\theta, x) := \sum_{c=1}^C \theta^{(c)} \star x_c$$

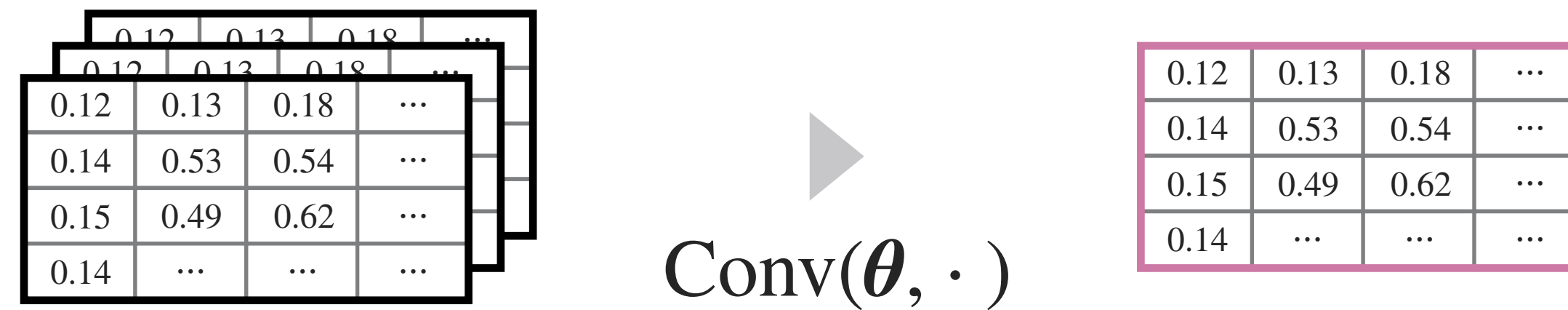
【補足】 もちろん、そもそもの畳み込みを、直方体の形のカーネルを動かしながら、入力の直方体領域との内積を計算してスキャンする操作として定義してもよい。

【用語】 2次元のものもカーネルと呼ばれるが、入力が複数チャネルある場合に直方体型にパラメーターを並べた3次元のものもカーネルと呼ばれる。

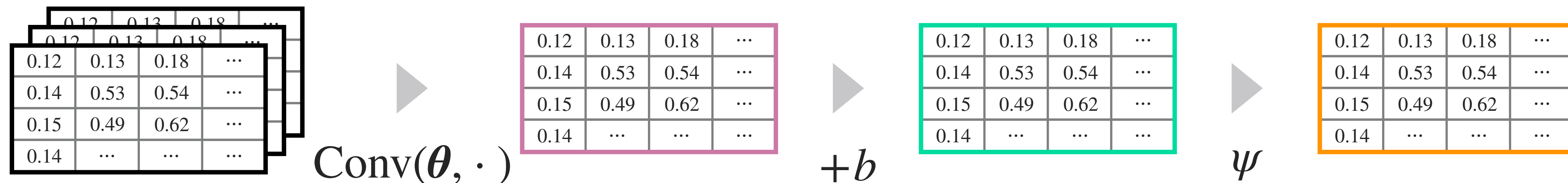
画像を扱うモデル＞畳み込み層

最後に、バイアスを足して活性化関数をかける

- この $\text{Conv}(\theta, \cdot)$ は、（複数チャネルの）画像入力から1チャネルの”画像”を出力



- 通常は、さらにバイアスを足して、活性化関数をかける



- 同じバイアス（実数 b ）の値を成分ごとに足す。 ψ も成分ごとに適用。

画像を扱うモデル > 畳み込み層

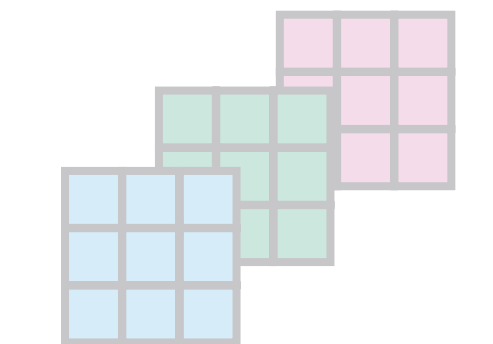
画像としての構造を活かしたまま処理する

- ここまでの内容はカーネル1つ分の話

0.12	0.13	0.18	...
0.12	0.13	0.18	...
0.14	0.53	0.54	...
0.15	0.49	0.62	...
0.14	...	...	...

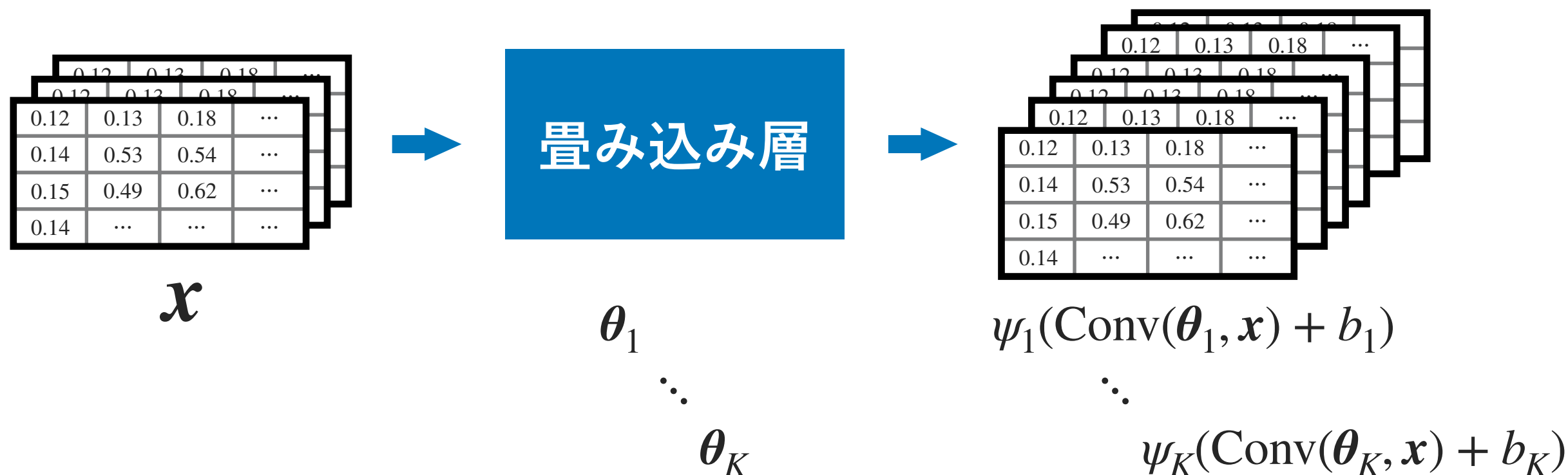
$$\psi(\text{Conv}(\theta, \cdot) + b)$$

0.12	0.13	0.18	...
0.14	0.53	0.54	...
0.15	0.49	0.62	...
0.14	...	...	...


$$\theta = (\theta^{(1)}, \theta^{(2)}, \theta^{(3)})$$

- 畳み込み層 (convolution layer)

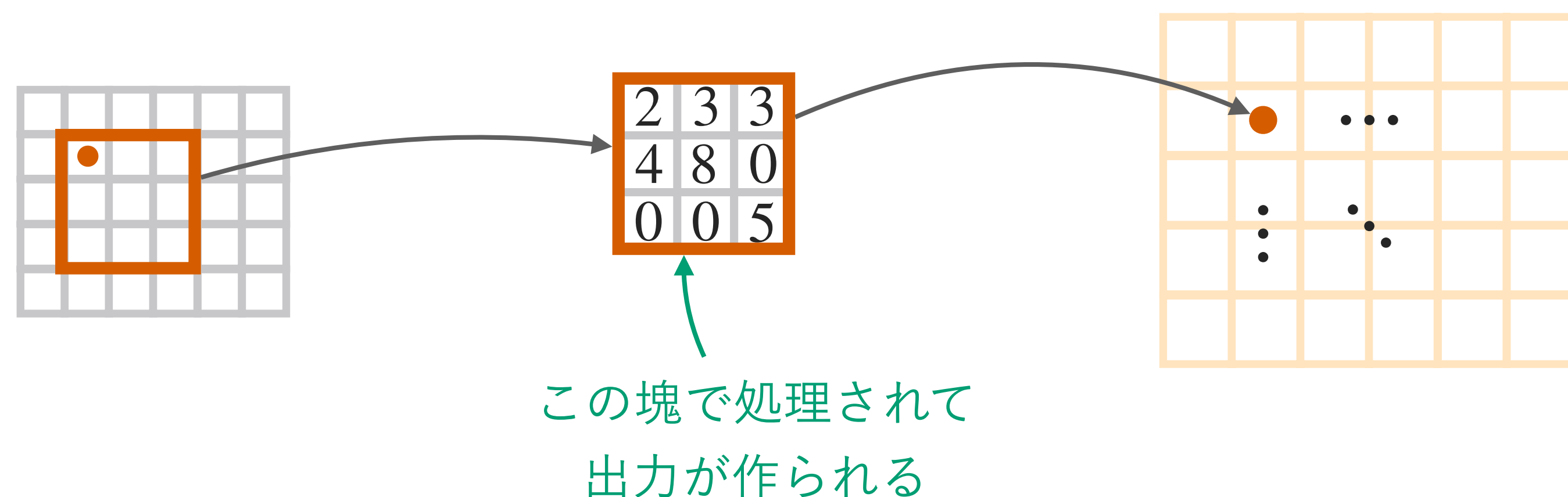
- 畳み込み層では、これらを好きな数だけ用意して、同様の処理を行う



画像を扱うモデル＞畳み込み層

数値の2次元・3次元的な配置をそのまま活かす処理になっている

- 数値の空間的な配置という、画像ならではの事前知識を活用
 - 2次元的に近い座標のピクセル値がまとめて処理され計算されていく

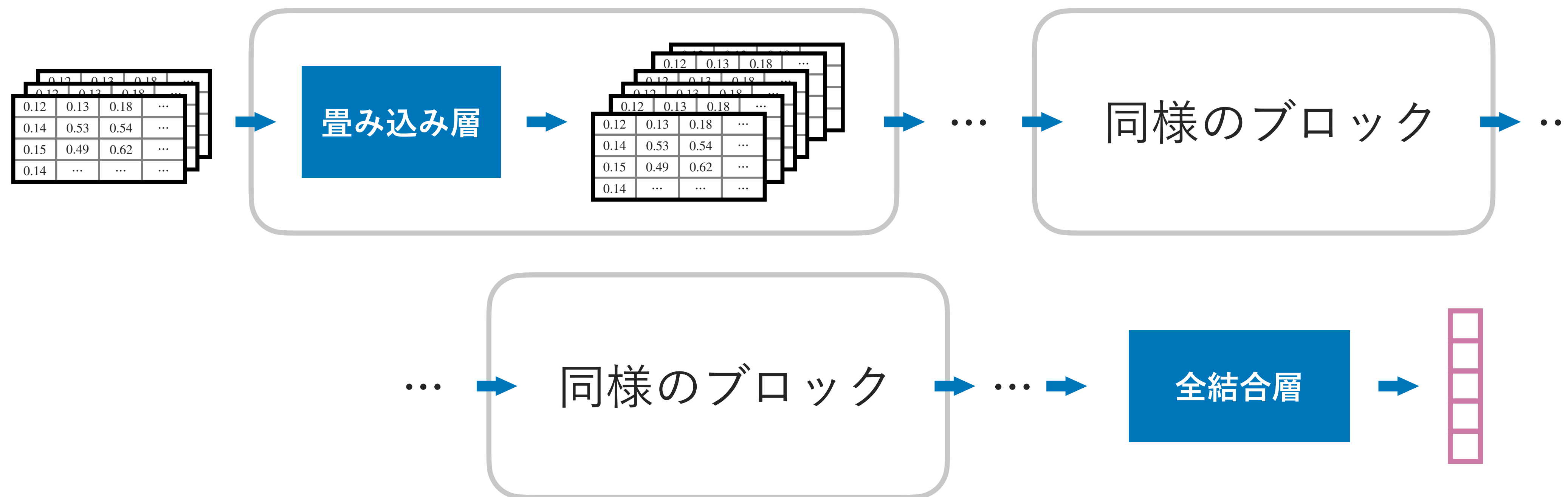


- フラット化して処理した場合は失われるような情報が、きちんと残ったまま処理

画像を扱うモデル > 畳み込み層 > 学習

畳み込みも，微分可能な関数になっている

- つまり，誤差逆伝播法で勾配を計算可能なので，畳み込み層を組み込んだアーキテクチャでも，勾配ベースの最適化法が使える



画像を扱うモデル＞まとめ



画像データに特化したモデルの主要な部品である「畳み込み層」を説明した

- 画像データにおける，数値の2次元・3次元的な配置をそのまま活かす処理
 - 数値の空間的な配置・形状という，画像ならではの事前知識を活用
 - 誤差逆伝播法で勾配を計算可能なので，ニューラルネットの部品として使える
- なお，畳み込み層と合わせて，以下のような層や工夫が使われる（省略）
 - グループ正規化層（group normalization）……平均を引いて分散で割る
 - プーリング層（pooling）……隣接するピクセル値を集計してサイズ圧縮
 - アテンション層（attention）……遠くのピクセル値の情報を混ぜ合わせる
 - スキップ接続（skip connection）……入力情報を維持して差分の計算に注力